

ResumeAI

AI-Powered Resume Analysis, Resume Builder, Interview Preparation and Job Matching System

A Project Report

Submitted in partial fulfilment of the requirements for the award of the degree of
[Degree Name — Not specified]

Submitted by
[Student Name — Not specified]

[Department / Institution Name — Not specified]
[Academic Year — Not specified]

Table of Contents

Chapter 1 — Introduction

1.1 Background

Job seeking is a document-heavy and repetitive process. A candidate typically needs a well-formatted resume, an understanding of how their resume compares against a target job description, and preparation for the interview that may follow. Traditionally, each of these steps is handled separately and manually — resumes are built in generic word processors, resume quality is judged subjectively, and interview preparation is done through generic question banks that are not tailored to the candidate's own background or the specific job being applied for.

ResumeAI is a web application built to bring these separate activities — resume creation, resume analysis, interview preparation, job matching, and application tracking — into a single system, using Large Language Model (LLM) based AI assistance where it adds genuine value, such as generating resume content, generating interview questions, and comparing a resume against a job description.

1.2 Problem Statement

Job seekers, particularly students and early-career professionals, commonly face the following difficulties:

- Creating a professionally formatted resume without design or word-processing expertise.
- Knowing whether a resume is well-written, complete, and appropriately structured before submitting it.
- Understanding how a resume is likely to be read by Applicant Tracking Systems (ATS) and recruiters.
- Preparing for interviews in a way that is specific to their own resume and the job description of the role they are applying for, rather than generic question lists.
- Judging how well a resume actually matches a particular job description before applying.
- Keeping track of which jobs have been applied to, and the status of each application.

ResumeAI addresses these problems by combining a resume upload and parsing pipeline, AI-assisted analysis and content generation, an interview question generator that uses both the resume and the job description, a job-matching feature, and an application tracking module, all accessible from a single dashboard.

1.3 Objectives

Primary Objective

To design and develop a web-based system that helps job seekers create, analyze, and improve resumes, prepare for interviews, and evaluate how well their resume matches a given job description, using AI/LLM assistance where relevant.

Secondary Objectives

- Allow users to upload existing resumes in PDF or DOCX format and have the content extracted automatically.
- Provide AI-based analysis and feedback on an uploaded resume.
- Provide both an AI-assisted resume builder and a manual resume builder, with selectable templates, theme colors, and fonts.

- Generate interview questions (with suggested answer guidance) based on a candidate's resume and a target job description, for a user-specified number of questions.
- Allow a user to compare a resume against a job description for a match assessment.
- Allow users to track jobs they have applied to.
- Provide a dashboard summarizing the user's resumes, applications, and interview preparation activity.

1.4 Scope

The scope of the project, as implemented, covers resume upload and parsing (PDF/DOCX), AI-based resume analysis, AI-assisted and manual resume building with template/theme/font selection, AI-generated interview questions driven by resume and job description context, AI-based job matching between a resume and job description, job application tracking, and a summary dashboard. The backend is built using Python and FastAPI, with SQLite as the database (accessed through SQLAlchemy) and the Groq API used as the LLM access layer. Frontend implementation details that could not be confirmed from provided project information are marked as Not specified throughout this report rather than assumed.

1.5 Motivation

AI language models are well suited to tasks that involve reading unstructured text (such as a resume) and producing structured, context-aware output (such as feedback, generated resume content, or interview questions). Building ResumeAI provided an opportunity to apply this capability to a genuinely useful, everyday problem — job search preparation — while also gaining practical, end-to-end experience in backend API development, database design, document parsing, and integrating a third-party LLM API into a real application.

Note: This report documents ResumeAI based on the feature description and technology stack provided for this report. Items that would normally be confirmed by inspecting source code (exact API endpoints, database schema, frontend framework) are explicitly marked "Not specified" rather than assumed, in line with the reporting instructions given for this document.

Chapter 2 — Existing System and Proposed System

2.1 Existing System

Conventionally, a job seeker builds a resume manually using a general-purpose word processor (such as Microsoft Word or Google Docs), often starting from a downloaded template. Resume quality is assessed informally, by the candidate themselves or by asking friends/mentors for feedback. Interview preparation is typically done by browsing generic, publicly available question lists that are not tailored to the candidate's resume or the specific job. Comparing a resume against a job description, when done at all, is usually a manual, visual comparison. Tracking job applications is commonly done in an ad-hoc way, such as a spreadsheet or not at all.

2.2 Limitations of the Existing (Manual) Approach

- No automated or structured feedback on resume content or completeness.
- No AI-assisted generation of resume content — the user must write everything themselves.
- Interview preparation is generic and not personalized to the candidate's actual resume or the job description.
- No systematic way to gauge resume-to-job-description fit before applying.
- No centralized tracking of job applications and their outcomes.
- Resume formatting/design work is repeated from scratch for every version of the resume.

2.3 Proposed System — ResumeAI

ResumeAI proposes a single web application that brings resume upload, AI-based resume analysis, AI-assisted and manual resume building (with templates, theme, and font selection), AI-generated interview preparation, AI-based job matching, and job application tracking together, backed by a FastAPI backend, a SQLite database, and the Groq API as the LLM access layer.

2.4 Advantages of the Proposed System

- AI-assisted resume content generation reduces the effort of writing a resume from scratch.
- AI-based resume analysis gives the user structured feedback rather than relying only on self-assessment.
- Interview questions are generated using both the resume and the job description, making preparation more relevant to the specific application.
- Job matching gives the user a way to assess fit before applying, rather than after.
- A single dashboard consolidates resumes, applications, and preparation activity in one place.

2.5 Existing System vs Proposed System — Comparison

Area	Traditional / Manual Approach	ResumeAI
Resume creation	Manual, in a general word processor	AI-assisted builder or manual builder with templates
Resume analysis	Informal / self-assessed	AI-based analysis on extracted resume text
AI assistance	None	LLM-based content generation and

Area	Traditional / Manual Approach	ResumeAI
		analysis via Groq API
Interview preparation	Generic question lists	Questions generated from the user's resume and job description
Job matching	Manual visual comparison, if done at all	AI-based comparison of resume and job description
Job tracking	Ad-hoc (spreadsheet or none)	Dedicated application tracking module
Customization	Limited to word processor formatting	Template, theme color, and font selection
Personalization	None	Content and questions generated per user resume/JD

Chapter 3 — Requirements Analysis

3.1 Functional Requirements

The system shall support the following functions:

1. The user can upload a resume in PDF or DOCX format.
2. The system can extract text content from an uploaded resume.
3. The user can request AI-based analysis of an uploaded resume.
4. The user can build a resume using the AI Resume Builder by selecting a template, theme color, and font, entering a resume title, and providing instructions to the AI.
5. The user can build a resume manually using the Manual Resume Builder, selecting a template, theme color, and font.
6. The system can generate resume content using AI based on user-provided instructions.
7. The user can preview the resume before saving.
8. The user can save the generated or manually built resume.
9. The user can print or download the resume.
10. The user can request interview questions by providing a resume, a job description, and the desired number of questions.
11. The system generates the requested number of interview questions, each with suggested approach/answer guidance.
12. The user can request a job-matching analysis by providing a resume and a job description.
13. The user can track jobs they have applied to.
14. The user can view a dashboard summarizing resumes, applications, and interview preparation activity.

3.2 Non-Functional Requirements

Requirement	Description	Status
Performance	The system should respond to user actions within a reasonable time; AI-dependent operations (analysis, generation, matching) are bound by third-party API response time.	Dependent on Groq API latency — Not benchmarked
Usability	The interface should allow a user with no technical background to upload, build, and manage resumes.	Design goal
Reliability	The system should handle invalid input (unsupported files, missing fields) gracefully.	Recommended improvement (see Error Handling, Chapter 8)
Maintainability	Code should be organized into routes, services, schemas, and database layers for ease of maintenance.	Reflected in the described backend folder structure
Security	API keys and sensitive configuration should not be exposed to the client.	Implemented via .env / python-dotenv for the API key; further hardening is a future

Requirement	Description	Status
		goal (see Chapter 8)
Scalability	The system should be able to handle a growing number of users and resumes.	Future goal — SQLite is not designed for large-scale concurrent production use
Availability	The system should be available whenever the backend server and database are running.	Applies to local/hosted deployment; no uptime guarantee implemented

3.3 Hardware Requirements

- A computer capable of running Python 3 and a modern web browser.
- Internet connectivity, required for the Groq API calls used by AI features.
- Exact minimum hardware specification: Not specified.

3.4 Software Requirements

- Python 3.x runtime for the backend.
- FastAPI framework and Uvicorn ASGI server.
- SQLite database engine.
- SQLAlchemy ORM.
- python-dotenv for environment variable management.
- Groq Python client, and a valid Groq API key.
- A PDF parsing library and a DOCX parsing library (exact libraries used: Not specified).
- Frontend runtime/framework: Not specified.

Note: Exact version numbers of the above libraries and the specific PDF/DOCX parsing libraries used were not provided and are therefore not stated as fact anywhere in this report.

Chapter 4 — Technology Stack

This chapter describes each technology actually used in the project, and explains, in simple terms, what role it plays. Only technologies confirmed for this project are described in detail; anything not confirmed is marked accordingly.

4.1 Python

Python is a general-purpose, high-level programming language known for readable syntax and a large ecosystem of libraries. It is used here to implement the entire backend of ResumeAI — the web server logic, file handling, database access, and the code that communicates with the Groq API.

4.2 FastAPI

FastAPI is a modern Python web framework used to build APIs. It allows routes (URL endpoints) to be defined as ordinary Python functions, automatically validates request and response data using type hints and Pydantic-style schemas, and automatically generates interactive API documentation. In ResumeAI, FastAPI is the framework that exposes the backend's functionality (upload, analyze, improve, interview, jobs) to the frontend over HTTP.

4.3 Uvicorn

Uvicorn is an ASGI (Asynchronous Server Gateway Interface) server used to actually run a FastAPI application — it is the process that listens for incoming HTTP requests and passes them to the FastAPI application code. FastAPI defines what happens when a request arrives; Uvicorn is what keeps the application running and reachable.

4.4 SQLite

SQLite is a lightweight, file-based relational database engine that does not require a separate database server process — the entire database lives in a single file (resume.db in this project). It is well suited to development, prototyping, and small-to-moderate scale applications because it requires no separate installation or server management, while still supporting standard SQL operations.

4.5 SQLAlchemy

SQLAlchemy is a Python Object-Relational Mapper (ORM). An ORM allows database tables to be represented as Python classes and rows as Python objects, so the application code can read and write data using normal Python syntax instead of writing raw SQL for every operation. This also makes the code more portable if the underlying database were ever changed.

4.6 Groq API and the Role of the LLM

Groq provides an API/inference platform through which a Large Language Model (LLM) can be accessed programmatically — Groq itself is not the AI model; it is the service that runs and exposes the model over an API, and is used here for its response-speed characteristics. The backend's AI service loads a GROQ_API_KEY from environment variables, creates a client using the Groq Python client library, and

sends prompts (system and user messages) to the LLM through this client. The specific LLM model name/version used through Groq: Not specified.

An LLM (Large Language Model) is a type of AI model trained on large amounts of text that can read an input prompt and generate relevant text as output — for example, reading a resume and job description and generating interview questions, or reading resume text and generating structured feedback. ResumeAI relies on the LLM's ability to read unstructured resume/job-description text and produce structured, relevant output.

4.7 PDF and DOCX Processing

The backend includes parsing logic for both PDF and DOCX resume files (referenced in the backend structure as `pdf_parser.py` and `docx_parser.py`), used to extract plain text content from an uploaded resume so it can be used by the AI-based features. The exact third-party parsing libraries used inside these modules: Not specified.

4.8 Environment Configuration

`python-dotenv` is used to load configuration values (such as the Groq API key) from a `.env` file into environment variables at runtime, keeping secrets out of the source code itself.

4.9 Frontend Technologies

The specific frontend framework/library used (e.g., a JavaScript framework, plain HTML/CSS/JS, or another approach) was not confirmed for this report. In the absence of project source files to inspect, this report does not assume a specific frontend technology such as React, Vue, or Angular. This should be filled in from the actual project source once available: Not specified.

4.10 Technology Stack Summary

Layer	Technology	Purpose
Backend language	Python	Implements the application logic
Web framework	FastAPI	Defines and exposes API routes
Application server	Uvicorn	Runs the FastAPI application
Database	SQLite	Stores application data
ORM	SQLAlchemy	Maps Python objects to database tables
AI / LLM access	Groq API (Groq Python client)	Sends prompts to an LLM and receives generated text
Configuration	<code>python-dotenv</code> (<code>.env</code>)	Loads secrets/configuration at runtime
Document processing	PDF parser, DOCX parser (modules present; specific libraries Not specified)	Extracts text from uploaded resumes
Frontend	Not specified	User interface

Note: Per the reporting instructions for this document, no frontend framework, specific parsing library, or LLM model version is assumed. These should be confirmed and filled in directly from the project's source files (e.g., requirements.txt, package.json) before final submission.

Chapter 5 — System Design

This chapter presents the major design views of ResumeAI. Each diagram is described in terms of its purpose and components, followed by a simple ASCII/text sketch that can be recreated in a diagramming tool such as draw.io, Lucidchart, or Mermaid.

5.1 System Architecture Diagram

Purpose: Shows the high-level layers of the system and how they connect — the frontend (client), the FastAPI backend, the service layer, the AI provider (Groq API), and the database.

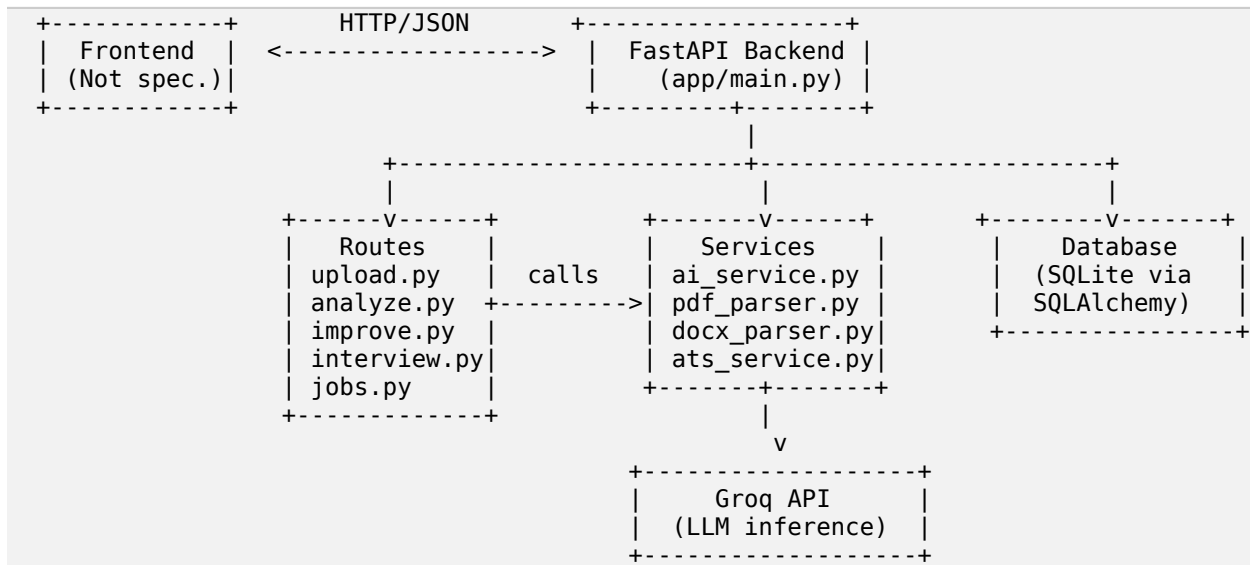


Figure 5.1 — System Architecture (conceptual)

5.2 Data Flow Diagram (Level 0 / Context)

Purpose: Shows how data moves between the user, the system, and external services at a high level.

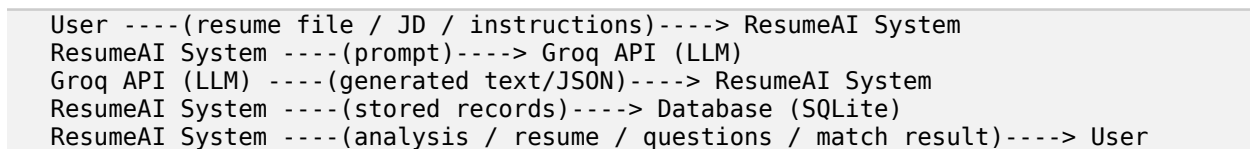
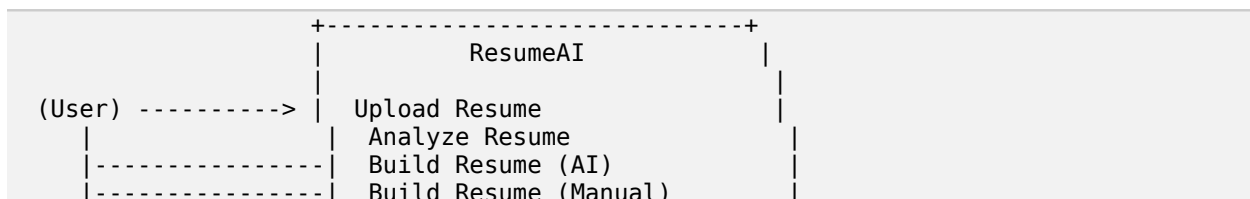


Figure 5.2 — Level 0 Data Flow Diagram

5.3 Use Case Diagram

Purpose: Shows the interactions between the user (actor) and the main system functions.



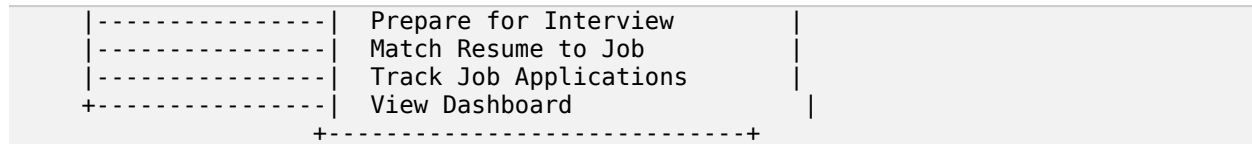


Figure 5.3 — Use Case Diagram

5.4 Component Diagram

Purpose: Shows the internal software components of the backend and how they depend on one another, based on the backend folder structure described in Chapter 6.

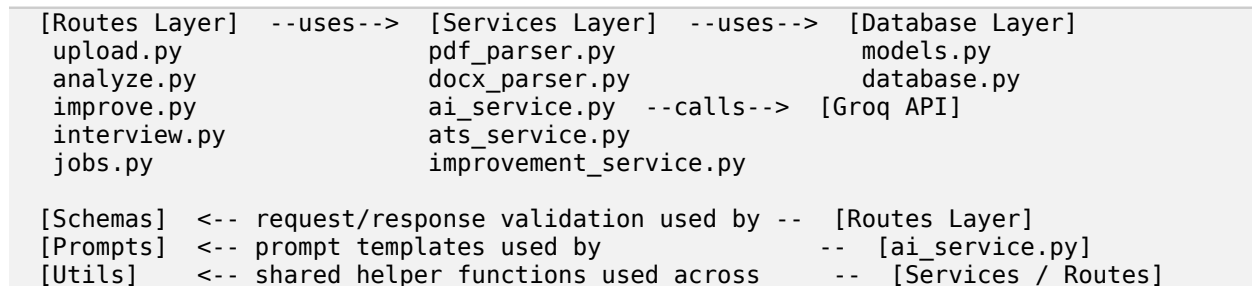


Figure 5.4 — Component Diagram

5.5 Database / ER Diagram

Purpose: Shows the entities managed by the application and their relationships, as far as can be inferred from the application's described features. Exact table/column names could not be confirmed without inspecting database/models.py — see Chapter 7 (Database Design) for details and limitations.

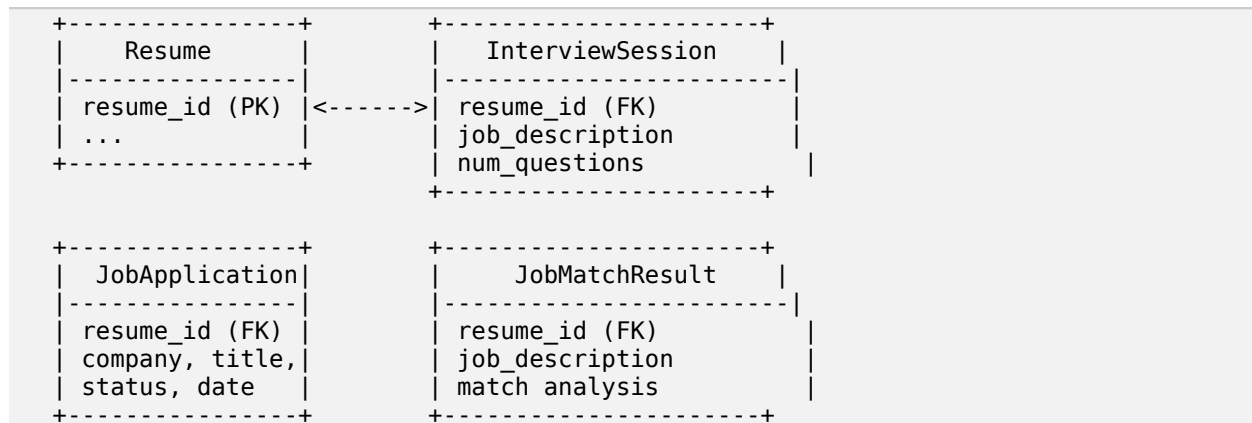


Figure 5.5 — Conceptual Entity Relationship sketch (fields not confirmed from source code — see Chapter 7)

Note: This ER sketch is conceptual, derived only from the described features (resume upload, interview preparation, job matching, job tracking). It is not based on inspection of database/models.py, since source code was not provided. Actual table names, columns, primary keys, and foreign keys must be confirmed from the SQLAlchemy models before this diagram is finalized.

5.6 Resume Processing Flowchart



Figure 5.6 — Resume Processing Flow

5.7 AI Resume Generation Flowchart

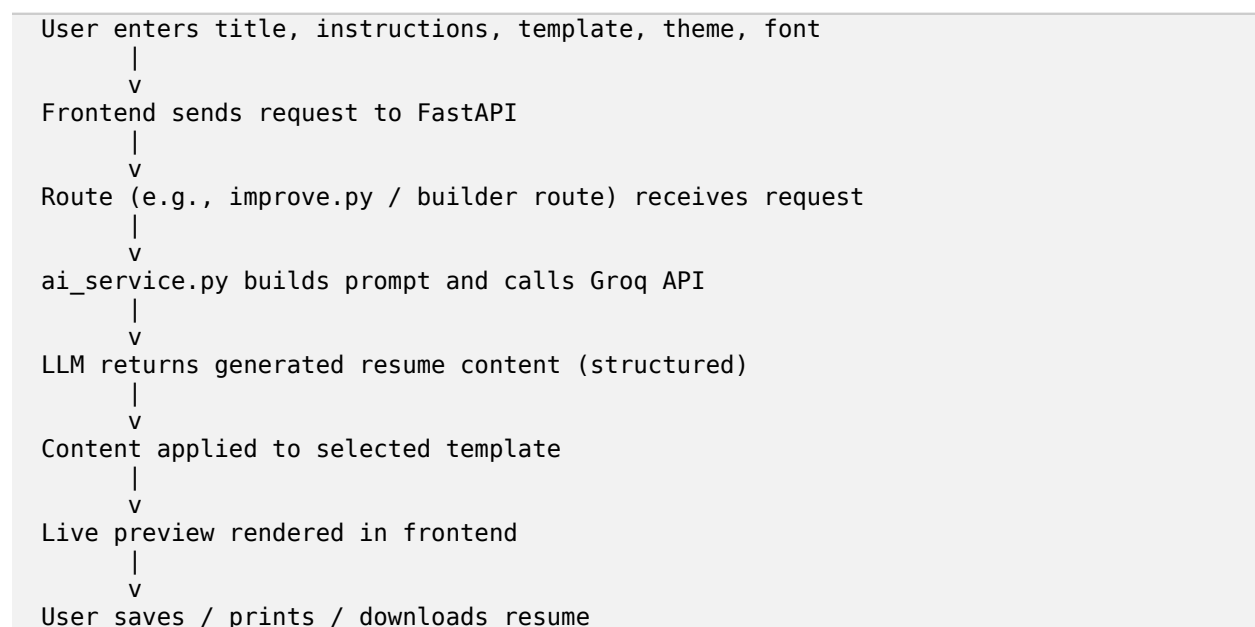
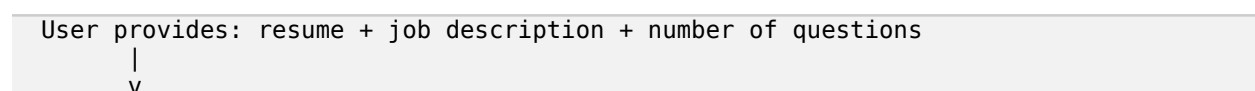


Figure 5.7 — AI Resume Generation Flow

5.8 Interview Preparation Flowchart



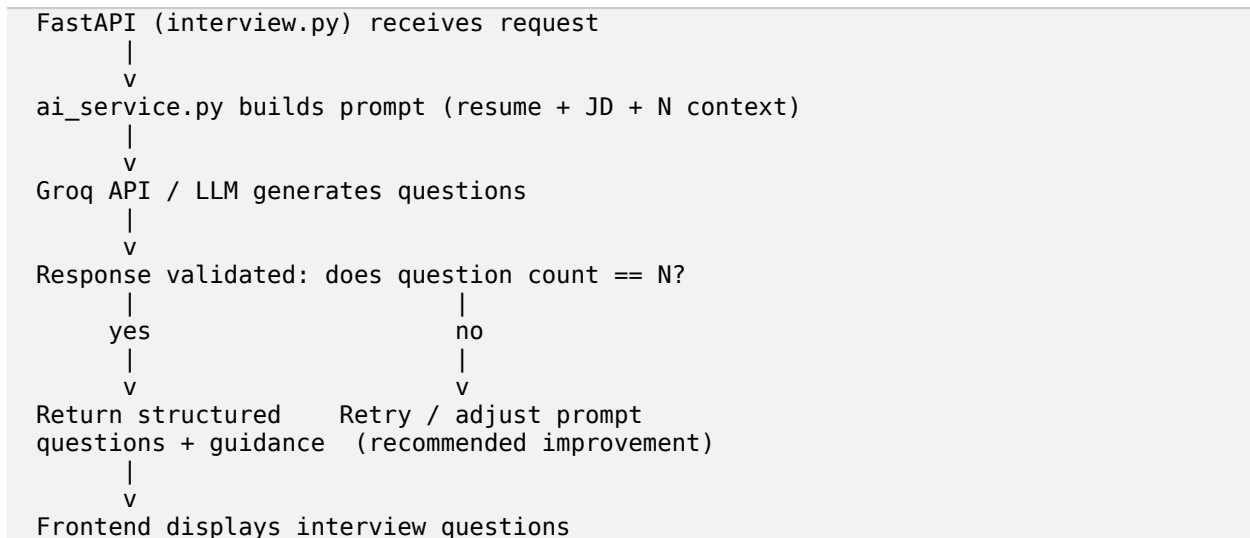


Figure 5.8 — Interview Preparation Flow

5.9 Job Matching Flowchart

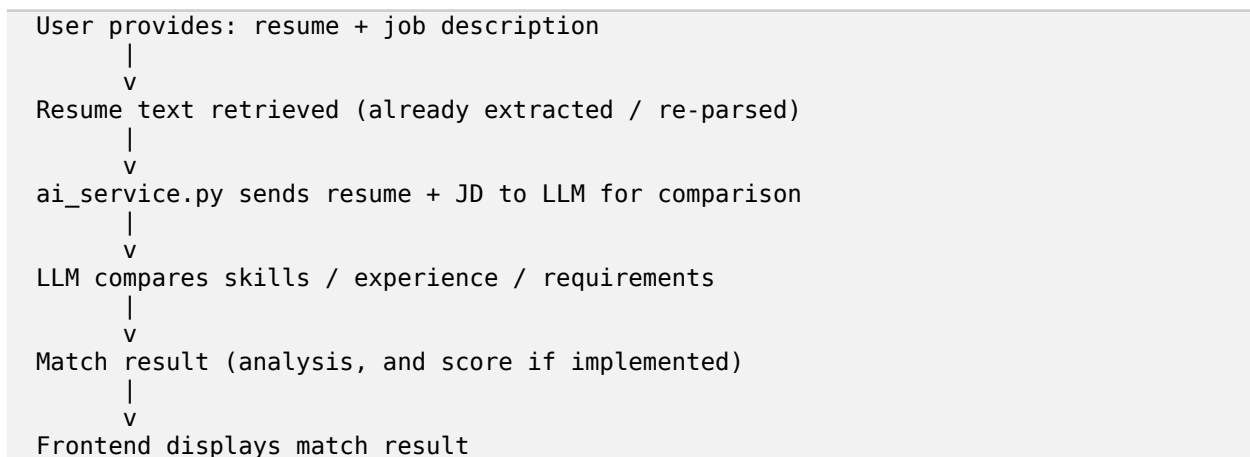


Figure 5.9 — Job Matching Flow

Chapter 6 — Implementation

This chapter describes how each feature is implemented, structured around the backend architecture. The backend folder structure, as described for this project, is shown below.

```

backend/
|
+-- app/
|   +-- main.py
|   |
|   +-- routes/
|       +-- upload.py
|       +-- analyze.py
|       +-- improve.py
|       +-- interview.py
|       +-- jobs.py
|   +-- services/
|       +-- pdf_parser.py
|       +-- docx_parser.py
|       +-- ai_service.py
|       +-- ats_service.py
|       +-- improvement_service.py
|   +-- database/
|       +-- models.py
|       +-- database.py
|   +-- schemas/
|   +-- utils/
|   +-- prompts/
+-- uploads/
+-- generated/
+-- resume.db
+-- requirements.txt
+-- .env
  
```

Figure 6.1 — Backend folder structure (as described)

Purpose of each module

Folder / File	Purpose
main.py	Entry point of the FastAPI application; registers routes and starts the app.
routes/	Defines the API endpoints (HTTP routes) that the frontend calls.
services/	Contains the business logic — parsing, AI calls, analysis — kept separate from the routing layer.
database/	Defines the SQLAlchemy models (tables) and the database connection/session setup.
schemas/	Defines request/response data shapes (e.g., Pydantic models) used for validation.
utils/	Shared helper functions used across the application.

Folder / File	Purpose
prompts/	Prompt templates used when constructing requests to the Groq API.
uploads/	Stores uploaded resume files.
generated/	Stores AI-generated or rendered resume output.
resume.db	The SQLite database file.
requirements.txt	Lists Python package dependencies.
.env	Stores environment variables such as the Groq API key (not committed to version control).

6.1 Resume Upload

The user uploads an existing resume in PDF or DOCX format through the upload interface. Resume upload is needed because it is the entry point for all AI-assisted features that require the candidate's actual background — without extracted resume content, the system has nothing specific to analyze, build from, or use for interview/matching context.

On the backend, the route in `upload.py` receives the file, and the appropriate parser (`pdf_parser.py` or `docx_parser.py`) is used depending on the file type to extract plain text. The uploaded file is stored (in the `uploads/` directory) and associated with a resume identifier so it can be referenced later by other features (analysis, interview preparation, job matching) without re-uploading. The extracted text becomes the input that AI features operate on.

6.2 Resume Parsing (PDF/DOCX Text Extraction)

PDF and DOCX files store text in structured, format-specific ways rather than as plain readable text. Parsing extracts the readable text content from these formats so it can be passed to the AI service as a plain string. The specific parsing libraries used inside `pdf_parser.py` and `docx_parser.py`: Not specified.

6.3 Resume Analysis

Once resume text has been extracted, the `analyze.py` route allows the user to request an AI-based analysis of the resume. The extracted text is passed to the AI service, which sends it to the LLM (via Groq) with a prompt requesting feedback. Aspects of analysis such as skills, experience, education, and projects can be covered because they are all present in typical resume text; the exact scope of the analysis prompt actually implemented (e.g., whether ATS-specific scoring is included) is handled by `ats_service.py` and `improvement_service.py` in the described structure — exact analysis criteria implemented: Not specified beyond what these module names indicate.

6.4 AI Resume Builder

The AI Resume Builder lets the user select a resume template, a theme color, and a font; enter a resume title; and provide instructions describing the resume content they want (e.g., career background, target role). This is sent to the backend, which uses `ai_service.py` to construct a prompt from the user's instructions and requests generated resume content from the LLM. The generated content is applied to

the selected template for a live preview, which the user can review before saving, printing, or downloading the resume.

Templates

The application includes multiple resume templates, including Classic Sidebar, Dark Banner, and Minimal. Template selection determines which layout is used to render the resume; the selected template is reflected in the live preview so the user sees the final appearance before saving.

6.5 Manual Resume Builder

The Manual Resume Builder allows the user to build a resume by entering content themselves rather than having it AI-generated, while still selecting a template, theme color, and font. The resume preview updates according to the selected configuration, using the same template rendering used by the AI builder.

Providing both an AI Resume Builder and a Manual Resume Builder is useful because it accommodates two different user needs: users who want AI assistance to draft content quickly, and users who prefer full control over their own wording but still want template-based formatting and a professional layout.

6.6 Template System

The template system separates resume content (data) from resume presentation (layout/styling). The same underlying resume data — whether AI-generated or manually entered — can be rendered using any of the available templates, with the selected theme color and font applied. This is what allows a live preview to update immediately when the user changes the template, theme, or font.

6.7 Interview Preparation

The user provides a resume, a job description, and the number of interview questions they want generated. The `interview.py` route passes this information to `ai_service.py`, which constructs a prompt combining resume context and job description context, and requests the LLM to generate that exact number of interview questions, each with suggested approach/answer guidance. Including the job description in the prompt — rather than generating generic questions — allows the questions to reflect the specific role's requirements, not just the candidate's general background.

Ensuring the system returns exactly the number of questions requested is an implementation requirement discussed further in Chapter 8 (Error Handling) and Chapter 10 (Known Issues) — LLMs do not always follow numeric instructions exactly, so response validation (and, if needed, a retry) is relevant here.

6.8 Job Matching

The user provides a resume and a job description. The system uses the AI service to compare the resume content against the job description, considering skills, experience, and other requirements mentioned in the job description, and returns an analysis of how well they match. Whether a specific numeric match score is generated (as opposed to a qualitative analysis) depends on the actual prompt/response design implemented — this report does not assume a specific scoring algorithm exists unless confirmed: Not specified.

6.9 Job Application Tracking

The Applied Jobs / Job Tracking module lets users record and track jobs they have applied to. Tracking applications in one place is useful because job searches typically involve applying to many roles over time, and it becomes difficult to remember which resume version, company, role, and status apply to each application without a dedicated record. Fields such as company, job title, status, and date are consistent with the feature description provided; exact fields implemented in the database should be confirmed from database/models.py — Not specified beyond what was described.

6.10 Dashboard

The dashboard provides a single overview screen summarizing the user's activity: resumes uploaded, resumes built, jobs applied to, interviews prepared, recent resumes, recent job applications, and quick actions to jump into a specific feature. Its role in the overall application is to reduce the need to navigate through each module individually just to see a summary of progress, and to serve as the natural landing page after login.

Chapter 7 — AI Integration

7.1 LLM Integration

ResumeAI integrates AI functionality through the Groq API, accessed via the Groq Python client inside `ai_service.py`. The API key (`GROQ_API_KEY`) is loaded from environment variables using `python-dotenv`, and a Groq client instance is created using this key. Every AI-dependent feature — resume analysis, AI resume content generation, interview question generation, and job matching — routes through this shared AI service rather than each feature implementing its own API calls, which keeps the prompt-building and API-handling logic centralized.

7.2 Prompt Engineering

Prompt engineering refers to designing the text sent to an LLM so that its response is accurate, relevant, and in the expected format. In ResumeAI, prompts are built from two parts: a system message that sets the LLM's role and instructions (for example, instructing it to act as a resume/interview assistant and to respond in a specific structure), and a user message that contains the actual context for the request — such as extracted resume text, a job description, or builder instructions.

Context Injection

- Resume context: extracted resume text is injected into the prompt for analysis, AI building, interview preparation, and job matching.
- Job description context: injected for interview question generation and job matching, so output is specific to the target role rather than generic.
- User instructions: injected for the AI Resume Builder, so generated content reflects what the user actually asked for.
- Question count: injected for interview preparation, instructing the LLM to generate a specific number of questions.

The `prompts/` folder in the backend structure indicates that prompt templates are kept as separate, reusable artifacts rather than being hard-coded inline throughout the service logic, which makes them easier to review and adjust independently of the application code.

7.3 Structured Output (JSON)

Where appropriate, the AI service requests structured JSON responses from the LLM rather than free-form text. This matters because the backend needs to programmatically use the LLM's output — for example, to populate specific resume template fields, to count how many interview questions were generated, or to display a job match analysis in a specific UI layout. Free-form text would need to be parsed unreliably after the fact; asking the LLM to directly return JSON in an agreed structure lets the FastAPI backend parse the response directly into Python objects and pass it on to the frontend.

7.4 Resume Analysis (AI Use)

For resume analysis, extracted resume text is sent to the LLM with a prompt asking for feedback relevant to resume quality (structure, completeness, clarity, and other resume-writing considerations, as covered in Chapter 6.3). The LLM's response is returned to the frontend as the analysis result.

7.5 Resume Generation (AI Use)

For AI-assisted resume building, the user's instructions (and any resume context, if provided) are sent to the LLM with a prompt requesting resume content organized in a way that can populate the selected template's fields (e.g., summary, experience, education, skills).

7.6 Interview Question Generation (AI Use)

For interview preparation, the resume text, job description, and requested question count are combined into a single prompt. The LLM is asked to generate that number of questions, each paired with suggested approach/answer guidance, ideally returned as a structured list so the frontend can render them individually.

7.7 Job Matching (AI Use)

For job matching, the resume text and job description are sent to the LLM with a prompt asking it to compare the two — identifying overlapping and missing skills, relevant experience, and other job requirements — and to return an analysis of the fit between the resume and the role.

7.8 Why an API-Based LLM Rather Than a Self-Hosted Model

Using an API-based LLM (via Groq) instead of hosting a model directly avoids the need to provision GPU infrastructure, manage model weights, or handle model serving, which is impractical for a student/academic project. It also allows the project to benefit from the capability of a larger, well-trained model than could realistically be trained or hosted independently.

Chapter 8 — Testing, Security, and Error Handling

8.0 Security Considerations

This section distinguishes between security measures that are implemented (as described for this project) and those that are recommended improvements but not confirmed as implemented.

Implemented

- API key protection: the Groq API key is kept in a .env file and loaded via python-dotenv, rather than being hard-coded into source files.

Recommended Improvements (not confirmed as implemented)

- File type validation on upload beyond basic extension/content checks (e.g., verifying actual file content matches the claimed type).
- File upload size limits and stricter upload security (e.g., scanning, sandboxed storage).
- Systematic input validation across all routes using schema validation for every field.
- Authentication and authorization (the project description does not mention a login/authentication system — see Chapter 11, Future Enhancements).
- Database-level security hardening (e.g., encryption at rest) for the SQLite file.
- Safe handling / sanitization of AI-generated output before it is rendered in the frontend, to avoid issues such as unexpected formatting or injection when displaying LLM output.
- Rate limiting on AI-dependent endpoints to control API usage and cost.

Note: No security features are described as implemented beyond .env-based API key storage. All other items above should be treated as recommended future work, not existing functionality, unless confirmed otherwise from source code.

8.1 Testing Strategy

As no test suite or test results were provided for this project, this section presents a recommended testing strategy — the test cases the project should be verified against — rather than reporting actual pass/fail results. This distinction is intentional, in line with the instruction not to fabricate performance or test metrics.

8.2 Test Cases

Resume Upload

Test Case	Input	Expected Result
Valid PDF upload	A well-formed .pdf resume file	File accepted, stored, and text extracted successfully
Valid DOCX upload	A well-formed .docx resume file	File accepted, stored, and text extracted successfully
Invalid file type	A file with an unsupported extension (e.g., .txt, .jpg)	Upload rejected with an appropriate error message
Missing file	Upload request sent with no file	Request rejected with a validation

Test Case	Input	Expected Result
	attached	error

Resume Analysis

Test Case	Input	Expected Result
Valid resume analysis	A valid resume_id for an uploaded resume	AI analysis returned successfully
Invalid resume ID	A resume_id that does not exist	Error returned indicating the resume could not be found
Empty resume	A resume file with little or no extractable text	Graceful handling — either a meaningful error or a low-confidence analysis, not a crash

AI Resume Builder

Test Case	Input	Expected Result
Valid instructions	Complete builder instructions, template, theme, font	Resume content generated and rendered in the selected template
Missing instructions	Empty or missing instructions field	Validation error, or a sensible default prompt behaviour
Template selection	Switching between Classic Sidebar, Dark Banner, Minimal	Preview updates to reflect the newly selected template
Theme selection	Changing theme color	Preview updates with the new theme color
Font selection	Changing font	Preview updates with the new font

Interview Preparation

Test Case	Input	Expected Result
Valid resume + JD	Valid resume_id and non-empty job description	Interview questions and guidance generated
Different question counts	Requesting, e.g., 5 vs 10 questions	Exactly the requested number of questions returned
Missing job description	Empty job description field	Validation error returned
Invalid resume	A resume_id that does not exist	Error returned indicating the resume could not be found
AI failure	Groq API unreachable or returns an error	Backend returns a handled error, not an unhandled crash

Job Matching

Test Case	Input	Expected Result
Valid resume + JD	Valid resume_id and non-empty job description	Match analysis returned
Invalid resume	A resume_id that does not exist	Error returned indicating the resume could not be found
Missing job description	Empty job description field	Validation error returned
AI failure	Groq API unreachable or returns an error	Backend returns a handled error, not an unhandled crash

Database

Test Case	Input	Expected Result
Record creation	Valid data for a resume/application/interview record	Record persisted and retrievable
Record retrieval	A valid record ID	Correct record returned
Invalid ID	A non-existent record ID	Handled error (e.g., 404), not an unhandled exception

8.3 Validation

Validation should occur at two levels: (1) request-level validation using FastAPI/schema validation to reject malformed requests before they reach business logic, and (2) response validation for AI-generated content — for example, checking that the number of interview questions returned matches the number requested before returning the response to the frontend.

8.4 Error Handling

Error Scenario	Recommended Handling	Status
Invalid resume file	Return a clear error indicating the file could not be processed	Recommended
Unsupported file format	Reject before parsing, with a specific error message	Recommended
Missing resume	Return a 404-style error for the given identifier	Recommended
Invalid resume ID	Return a 404-style error rather than an unhandled exception	Recommended
Empty job description	Reject at validation stage with a clear message	Recommended
Missing API key	Fail fast at startup / on first AI call with a clear configuration error	Recommended
AI API errors (Groq unreachable/rate-limited)	Catch the exception and return a handled error response, not a raw stack trace	Recommended

Error Scenario	Recommended Handling	Status
Invalid/malformed AI response (not valid JSON)	Validate the response before using it; return an error if parsing fails	Recommended
Database errors	Catch exceptions from the ORM layer and return a handled error	Recommended
Incorrect number of generated interview questions	Validate count against the request; retry or trim/pad with a clear note if mismatched	Recommended — also discussed in Chapter 10

Note: The table above lists recommended error-handling behaviour. Whether each of these is already implemented in the codebase was not confirmed and should be verified against the actual route/service code before this section is finalized.

Chapter 9 — Results and Discussion

9.1 Features Implemented

The following features are described as implemented in ResumeAI:

- Resume upload (PDF and DOCX) with text extraction.
- AI-based resume analysis.
- AI Resume Builder with template, theme color, and font selection, AI-generated content, live preview, save, and print/download.
- Manual Resume Builder with template, theme color, and font selection.
- Interview Preparation — AI-generated questions with suggested guidance, based on resume and job description, for a user-specified number of questions.
- Job Matching — AI-based comparison of resume and job description.
- Job Application Tracking (Applied Jobs).
- Dashboard summarizing resumes, applications, and interview preparation activity.

9.2 User Interface — Screens

The application is organized around the following main screens.

Screen	What the User Can Do
Dashboard	View a summary of resumes uploaded/built, jobs applied, interviews prepared, recent resumes, recent applications, and access quick actions.
Resume Analysis	Upload or select a resume and request AI-based analysis and feedback.
Resume Builder (Manual)	Manually enter resume content, choose a template, theme color, and font, and preview the result.
AI Builder	Provide a title and instructions to the AI, choose a template, theme, and font, generate content, preview, save, and print/download.
Interview Preparation	Provide a resume, a job description, and the number of questions; view generated questions and guidance.
Job Matching	Provide a resume and a job description; view the AI-generated match analysis.
Applied Jobs	View and manage a list of jobs the user has applied to.

Note: This report describes each screen's intended purpose based on the feature list provided. Actual screenshots should be inserted here from the running application before final submission; none are included in this generated report since no application screenshots were provided.

9.3 AI Functionality — Discussion

The AI functionality in ResumeAI is used for four distinct tasks: analyzing resume content, generating resume content, generating interview questions, and comparing a resume against a job description. In each case, the underlying mechanism is the same — extracted or user-provided text is combined into a prompt and sent to the LLM via the Groq API, and the response is used to drive the corresponding feature.

Centralizing this through a single `ai_service.py`, as described in the backend structure, means the quality of every AI feature depends heavily on prompt design (see Chapter 7) rather than on separate, feature-specific AI logic.

9.4 Observations

- Combining resume and job-description context for interview questions is what differentiates this system's interview preparation from generic, non-personalized question banks.
- Because there is no confirmed authentication system, resume/application data as currently described appears to be identified by record/resume identifiers rather than tied to individual user accounts — this is discussed further as a limitation and future enhancement.
- The reliance on structured JSON responses from the LLM (Chapter 7.3) means the reliability of features such as exact interview-question counts depends on how strictly the LLM follows the requested format, which is a known characteristic of LLM-based systems (see Chapter 10).

Chapter 10 — Challenges and Limitations

10.1 Challenges Faced During Development

- PDF/DOCX text extraction: resumes come in widely varying layouts (columns, tables, graphics-heavy designs), which makes reliable text extraction harder than parsing plain text documents.
- Unstructured resume data: extracted resume text has no guaranteed structure (no consistent section headers or ordering), which the AI service has to work with as free text.
- LLM output consistency: LLM responses can vary between calls even for similar prompts, which affects features that expect a specific format or count (e.g., interview question count).
- Prompt design: crafting prompts that reliably produce the intended structure (e.g., valid JSON, exact question counts) required iteration.
- Structured JSON output: ensuring the LLM's response can always be parsed as valid JSON, and handling cases where it cannot.
- File handling: safely accepting, validating, and storing uploaded files of different formats.
- Resume-template rendering: applying AI-generated or manually entered content consistently across multiple templates (Classic Sidebar, Dark Banner, Minimal).
- Maintaining frontend state: keeping the resume preview, selected template/theme/font, and generated content in sync across the builder interfaces.
- Connecting frontend and backend: coordinating request/response formats between the frontend and the FastAPI routes.
- Database persistence: ensuring resume, interview, and job-tracking records are correctly saved and retrievable via SQLAlchemy.
- API errors and validation: handling failures from the Groq API and validating incoming requests consistently across routes.

10.2 Known Issues / Lessons Learned

The following practical issues are typical of a project with this feature set, and are documented here as development lessons along with possible solutions.

15. Resume IDs being difficult for users to identify when filenames are UUID-based — Lesson: expose a human-readable resume title/label in the UI alongside the internal identifier, rather than showing the raw ID to users.
16. Resume ID fields in Interview Preparation / Job Matching needing better integration with uploaded resumes — Lesson: let users pick from a list of their previously uploaded/built resumes rather than requiring them to know or re-enter an ID.
17. UI state resetting after AI generation — Lesson: preserve builder selections (template, theme, font, instructions) in frontend state across the generation request/response cycle.
18. AI returning fewer interview questions than requested — Lesson: validate the returned count against the requested count server-side, and retry or explicitly prompt the LLM again if the count does not match (see Chapter 8.4).
19. Template preview needing to accurately show the selected template — Lesson: ensure the preview rendering path and the final saved/printed rendering path use the same template-rendering logic, so what the user previews is what they get.
20. Template preview and generated resume needing consistent rendering — Lesson: treat resume data and template rendering as separate concerns (see Chapter 6.6) so any content — AI-generated or manual — renders identically in preview and output.

21. Need for frontend/backend state synchronization — Lesson: rely on the backend as the source of truth for saved resume state, refetching after save rather than assuming frontend state matches backend state indefinitely.

10.3 Limitations

- AI output (analysis, generated resume content, interview questions, match analysis) may not always be accurate, complete, or ideally phrased — it reflects the LLM's interpretation of the input, not a guaranteed-correct assessment.
- Resume parsing may fail or produce incomplete text for resumes with unusual layouts, heavy use of graphics, or non-standard formatting.
- LLM responses can vary between runs, even for very similar prompts, which can affect consistency of generated content.
- The depth of ATS-related analysis depends entirely on what is actually implemented in `ats_service.py` — this report does not assume a specific ATS scoring methodology beyond the module's presence.
- Job matching quality depends heavily on the quality and specificity of both the resume text and the job description provided by the user.
- SQLite, while suitable for development and small-scale use, is not designed for high-concurrency, large-scale production deployments.
- The system depends on the availability and responsiveness of the Groq API; if the API is unreachable or rate-limited, AI-dependent features will not function.

Chapter 11 — Future Enhancements

The items below are proposed future improvements. None of them are implemented in the current system as described; they are listed here as directions for future work.

- Authentication and user accounts, so resumes, applications, and interview sessions are tied to individual users.
- Better resume management, including resume versioning and history.
- Improved, more transparent ATS scoring.
- Additional resume templates.
- A drag-and-drop resume editor.
- More accurate and robust resume parsing for varied layouts.
- Semantic (embedding/vector-based) resume-to-job-description matching, if greater matching accuracy is needed than direct LLM comparison provides.
- Job portal integration, to pull job descriptions directly rather than requiring manual entry.
- Automated job recommendations based on a user's resume and history.
- Interview simulation (e.g., timed or conversational practice).
- Voice-based interview practice.
- A more detailed analytics dashboard.
- Migration to a cloud-hosted database for multi-user, production-scale use.
- A formal production deployment (see Chapter 12).
- Improved PDF generation quality for downloaded resumes.
- Resume history/version management across edits.

Chapter 12 — Deployment

This chapter discusses how ResumeAI could be deployed. As of this report, the project is not confirmed to be deployed to a live/production environment — this section describes deployment considerations, not a completed deployment.

11.1.1 Local Development

In local development, the FastAPI backend is run using Uvicorn, reading configuration (including the Groq API key) from a local `.env` file, and using the local `resume.db` SQLite file for storage. The frontend (technology not specified) would be run separately and configured to call the local backend's API base URL.

11.1.2 Backend Server

For a hosted deployment, the FastAPI application would need to run behind an ASGI server (Uvicorn, optionally behind a process manager or reverse proxy such as Nginx) on a server or cloud compute instance.

11.1.3 Environment Variables

In any deployment, the Groq API key and other configuration values should continue to be supplied via environment variables (or a secrets manager in a cloud environment) rather than committed to source control.

11.1.4 Database

SQLite is appropriate for local development and small-scale use. For a genuine multi-user production deployment, migrating to a server-based database (e.g., PostgreSQL or MySQL) would be a reasonable future step, since SQLite's file-based, single-writer design does not scale well to concurrent multi-user production traffic.

11.1.5 Frontend Hosting

Frontend hosting approach: Not specified, since the frontend technology itself was not confirmed for this report.

11.1.6 API Hosting

The FastAPI backend could be hosted on any platform capable of running a Python/Uvicorn process (e.g., a cloud VM, a container platform, or a Platform-as-a-Service offering). Specific hosting provider used: Not specified — none was confirmed for this project.

11.1.7 Production Considerations

- Migrating from SQLite to a production-grade database for concurrent multi-user access.
- Adding authentication before exposing the system publicly, since job/resume data is personal.
- Adding rate limiting and monitoring around Groq API usage and cost.
- Implementing the error-handling and input-validation improvements listed in Chapter 8.

- Running the backend behind HTTPS in any public deployment.

Note: No claim is made that ResumeAI is currently deployed to production; this chapter describes deployment considerations only.

Chapter 12 — Learning Outcomes

Building ResumeAI required applying a broad range of software development skills in an integrated, end-to-end way. Key learning outcomes include:

- Practical Python programming applied to a real, multi-feature backend application.
- Building and structuring a REST API using FastAPI, including route design and request/response handling.
- Understanding and applying core REST API concepts (endpoints, HTTP methods, JSON request/response bodies, status codes).
- Designing a layered backend architecture that separates routes, services, schemas, and the database layer.
- Using SQLAlchemy as an ORM, and understanding how it maps Python objects to relational database tables.
- Working with SQLite as a lightweight relational database suitable for development-scale projects.
- Considerations in relational database design, such as identifying entities and relationships (see Chapter 5.5 and Chapter 7).
- File handling in a backend context — accepting, validating, and storing uploaded files.
- Extracting text from PDF and DOCX documents programmatically.
- Integrating a third-party LLM API (Groq) into an application, including authentication via API keys and environment variables.
- Prompt engineering — designing system/user prompts, and using context injection to make LLM output relevant to a specific resume and job description.
- Requesting and working with structured JSON output from an LLM, and the reasons this matters for backend integration.
- Coordinating frontend and backend integration, including keeping UI state (template/theme/font selections) synchronized with backend-driven content.
- Basic UI/UX considerations for a multi-screen application (dashboard, builders, analysis, interview prep, matching, tracking).
- State management within a frontend application, particularly for live-updating previews.
- Designing error handling for a range of failure scenarios, including third-party API failures.
- Planning a testing strategy and writing meaningful test cases for a feature-rich application.
- General debugging skills across a full-stack, AI-integrated application.
- Broader software architecture thinking — separating concerns across layers so the system remains maintainable as features are added.
- Practical experience in AI application development, specifically applying LLMs to real, non-trivial tasks rather than toy examples.

Chapter 13 — Conclusion

ResumeAI brings together resume upload and parsing, AI-based resume analysis, AI-assisted and manual resume building with template/theme/font customization, AI-generated interview preparation driven by resume and job-description context, AI-based job matching, and job application tracking, within a single FastAPI-based backend and SQLite database, using the Groq API as the LLM access layer.

The project demonstrates how an LLM can be applied to a genuinely useful, everyday problem — job search preparation — by using it not as a generic chatbot, but as a component that reads specific, user-provided context (a resume, a job description, builder instructions) and produces structured, task-specific output. This required careful attention to prompt design and structured (JSON) output, in addition to standard backend engineering practices such as API design, database modeling, and file handling.

From a technical-skills perspective, the project involved practical experience across backend API development (FastAPI/Uvicorn), database design and access (SQLAlchemy/SQLite), document processing (PDF/DOCX parsing), third-party AI API integration (Groq), and the software architecture discipline of separating routes, services, and data access layers.

As documented in this report, several aspects of the system — exact database schema, exact API endpoints, the specific frontend technology used, and confirmed security/testing status — could not be verified without direct inspection of the project's source code, and are marked accordingly rather than assumed. The features described, however, represent a coherent and practically useful application of AI to resume preparation and job search readiness, with clear directions for future enhancement (Chapter 11), particularly around authentication, more advanced matching techniques, and production-readiness.

References

The following are general references relevant to the technologies used in this project. Specific citations (e.g., exact library versions, official documentation links used during development) were not provided and are not fabricated here.

- FastAPI official documentation — <https://fastapi.tiangolo.com/>
- Uvicorn official documentation — <https://www.uvicorn.org/>
- SQLAlchemy official documentation — <https://www.sqlalchemy.org/>
- SQLite official documentation — <https://www.sqlite.org/>
- Groq API / Groq Python client documentation — <https://groq.com/>
- python-dotenv documentation — <https://pypi.org/project/python-dotenv/>
- Python official documentation — <https://docs.python.org/3/>

Note: Add any additional references actually used during development (tutorials, Stack Overflow threads, specific parsing/rendering libraries once confirmed) before final submission.

Appendix A — Database Design (Detailed)

The project uses SQLite as its database engine, accessed through the SQLAlchemy ORM, with SQLAlchemy model classes defined in `database/models.py` and the database connection/session configured in `database/database.py`.

A.1 What SQLite Provides Here

SQLite stores the entire database as a single file (`resume.db`). This is well suited to this project because it needs no separate database server process, which simplifies local development and small-scale deployment, while still supporting standard relational operations (tables, primary keys, foreign keys, queries) through SQLAlchemy.

A.2 What SQLAlchemy Provides Here

SQLAlchemy allows each database table to be defined as a Python class (a "model"), with each row represented as an instance of that class. This means the rest of the backend code (routes, services) can create, read, update, and query records using normal Python objects and method calls, rather than writing raw SQL statements throughout the codebase. This also makes validation, relationships between tables, and future schema changes easier to manage centrally in `database/models.py`.

A.3 Exact Tables, Columns, and Relationships

The exact table names, column names, data types, primary keys, and foreign key relationships are defined in `database/models.py`, which was not provided for inspection when this report was generated. Rather than inventing table/column names, this report lists the entities that the described features imply must be persisted in some form, and marks the schema itself as not specified.

Entity (implied by features)	Likely Purpose	Exact Schema
Resume record	Associates an uploaded or built resume with a resume identifier, its extracted/generated content, and metadata (e.g., template, theme, font).	Not specified — confirm from <code>models.py</code>
Interview preparation record	Associates a resume and job description with generated interview questions and the requested question count.	Not specified — confirm from <code>models.py</code>
Job match record	Associates a resume and job description with a match analysis result.	Not specified — confirm from <code>models.py</code>
Job application (tracking) record	Stores tracked job applications (e.g., company, title, status, date, as described in the feature list).	Not specified — confirm from <code>models.py</code>

Note: This appendix should be replaced with the actual table definitions (column names, types, keys, relationships) extracted directly from `database/models.py` once the source code is available, per the reporting instructions for this document.

Appendix B — API Documentation (Template)

The backend exposes its functionality through FastAPI routes, organized by feature: `upload.py`, `analyze.py`, `improve.py`, `interview.py`, and `jobs.py`. The exact endpoint paths, HTTP methods, request bodies, and response shapes were not provided for inspection, so this appendix provides a template — one row per described feature — to be completed directly from the actual route definitions rather than invented endpoint names.

Feature (route file)	Likely Method	Purpose	Endpoint / Request / Response
Resume upload (<code>upload.py</code>)	POST	Upload a resume file (PDF/DOCX) and store it	Not specified — confirm exact path and schema from source
Resume analysis (<code>analyze.py</code>)	POST / GET	Request AI-based analysis of a stored resume	Not specified — confirm exact path and schema from source
Resume improvement / AI generation (<code>improve.py</code>)	POST	Generate or improve resume content using AI	Not specified — confirm exact path and schema from source
Interview preparation (<code>interview.py</code>)	POST	Generate interview questions from a resume, job description, and question count	Not specified — confirm exact path and schema from source
Job matching / tracking (<code>jobs.py</code>)	POST / GET	Compare a resume to a job description, and/or manage tracked job applications	Not specified — confirm exact path and schema from source

For each actual endpoint, the final report should document: the exact endpoint path, HTTP method, purpose, request parameters/body schema, response schema, and possible error responses — inspected directly from the route and schema definitions.

Appendix C — Technical Skills Demonstrated

Skill	How It Was Used in the Project
Python	Implemented the entire backend application logic.
FastAPI	Defined and exposed the application's API routes.
REST API design	Structured feature access (upload, analyze, improve, interview, jobs) as HTTP endpoints.
SQLAlchemy	Mapped application data (resumes, interviews, job records) to database tables as Python models.
SQLite	Provided the file-based relational database for the application.
Groq API	Provided programmatic access to an LLM for analysis, generation, interview questions, and matching.
LLM usage	Applied an LLM to read resume/job-description text and produce structured, task-specific output.
Prompt engineering	Designed system/user prompts with resume, job description, and instruction context injection.
PDF parsing	Extracted text content from uploaded PDF resumes.
DOCX parsing	Extracted text content from uploaded DOCX resumes.
JSON	Used as the structured format for AI responses and API request/response bodies.
Environment variables (.env)	Kept the Groq API key and configuration out of source code.
Database design	Structured resume, interview, and job-tracking data as related entities (see Appendix A).
Frontend integration	Connected the frontend to backend endpoints for all features (frontend technology not specified).
File handling	Managed upload, storage, and retrieval of resume files.
AI application development	Built multiple LLM-driven features within a single coherent application.
Debugging	Diagnosed and resolved issues such as those listed in Chapter 10 (e.g., mismatched interview-question counts).
Testing (strategy)	Defined test cases across upload, analysis, building, interview prep, matching, and database operations (Chapter 8).